

Machine Learning - Project 1

Yasmine Chaker, Anders Mickael Jacques Hominal, Nicolas Filimonov
EPFL Machine Learning and Optimization Laboratory, Lausanne, Switzerland
30 October 2023

I. INTRODUCTION

Cardiovascular Diseases (CVDs), such as heart attacks, are a leading global cause of death. We are utilizing machine learning on data from the Behavioral Risk Factor Surveillance System (BRFSS) to predict the risk of Myocardial Infarction or Coronary Heart Disease (MICHHD) based on lifestyle factors. Our objective is to develop an effective model for early MICHHD detection and prevention. This report outlines our data analysis, feature engineering, machine learning implementation, model evaluation, and predictions. It emphasizes the potential of Machine Learning in mitigating CVD risks, regardless of one's medical background knowledge.

II. METHODOLOGY

Firstly, we perform data pre-processing and feature selection to enhance the results of the model. Following that, we implement six algorithms: Mean Squared Error with Gradient Descent, Mean Squared Error with Stochastic Gradient Descent, Least Squares, Ridge Regression, Logistic Regression, and Regularized Logistic Regression. To assess the models, we also utilize a combination of 5-fold Cross Validation to prevent over-fitting on any subpart of the training data and a 0.8 / 0.2 training / testing ratio split in data. We evaluate the results using F1 score (1) and Accuracy (2). We will define these terms later on.

III. DATA PRE-PROCESSING AND FEATURE SELECTION

A. Feature Selection

The dataset comprises a training set and a test set. They respectively contain 328135 and 109379 rows each corresponding to a contacted person, and both contain 321 features. We conduct an exploratory data analysis to gain an initial understanding of the dataset and its structure. We remove certain features due to their irrelevance within the context of this project, namely: the date of the interview of the patient, the internal state ID in the study, and all the additional metadata features.

After browsing the [features codebook](#), we conclude that the first 27 columns fit this pattern. Following this initial selection step, the number of features decreases to 294. Secondly, we also clean the data. The first step consists of evaluating the number of NaN (Not a Number) entities populating the dataset. We observe in Figure 1 that there are 149 features (46.56% of all features) that contain 50% NaN or more.

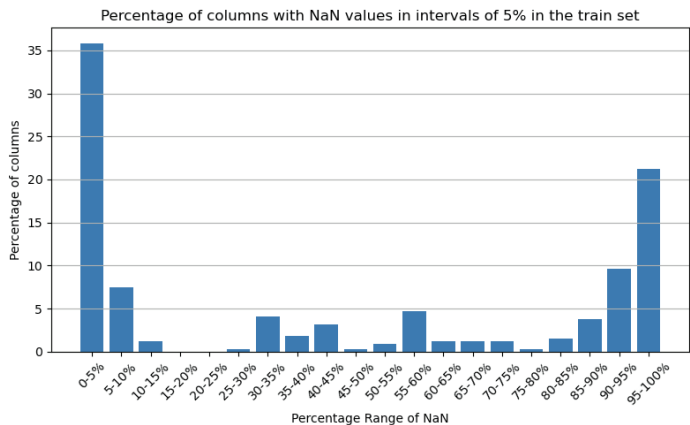


Figure 1: Percentage of columns with NaN values in intervals of 5% in the dataset

After dropping initially the columns that contains over 50% NaN and replacing the missing values by zeros, we realise that we are losing too much information, as we are replacing mostly the [calculated variables](#). Thus, we decide (in order to optimize the F1 score) to:

- Remove the features that contain 97% or more of NaN, dropping the number of features to 251.

- Replace the remaining NaNs by the mean of their column.

B. Data Pre-processing

Secondly, after an initial exploration of the above-mentioned codebook, we notice that the dataset is full of candidates for outliers. For example, numbers such as 99 or 999 (for missing responses as default error code), are observed in some columns. To eliminate the extreme values, we use the interquartile range (IQR) method to filter out potential outliers. The IQR method identifies values outside the 25th and 75th percentiles. We then, after testing, decide to replace them with the mean of the column and not by 0 for optimization purposes. In Figure 2, we can see the ratio of outliers in each column. Since it appears that there is indeed a lot of them, it justifies the method.

Additionally, we apply logarithmic transformation to decrease the variability of features and make data conform more closely to a normal distribution.

Finally, we standardize by subtracting the mean and dividing by the standard deviation. This transformation ensures that each feature contributes equally to the distance computation, ensuring that the algorithm works correctly. We will show this later in this report.

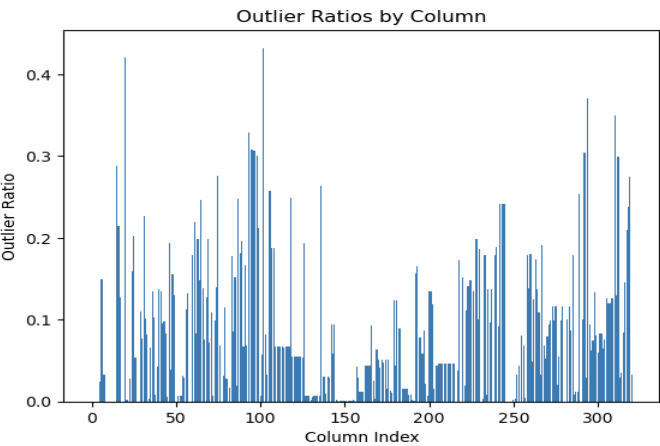


Figure 2: Outlier candidate Ratio for each feature

IV. RESULTS

A. Dataset Splitting

To evaluate the model, we use a combination of a 5-fold Cross Validation and a 80% 20% training / testing ratio split in data.

We use the cross-validation method to compare and to generate the best hyperparameters of our models. It does this by training and evaluating the model on multiple different folds. The best hyperparameters are the ones that produce the highest f1 score on the validation set. In more detail, the function works as follows:

- 1) The data is split into $k=5$ folds. This means that the data is divided into 5 equal parts, and each part is used as a validation set once.
- 2) Variables are initialized to store the best hyperparameters and corresponding results. The best hyperparameters are the ones that produce the highest f1 score on the validation set.
- 3) The function loops over all combinations of hyperparameters. For each combination of hyperparameters, the model is trained on the training set and evaluated on the validation set. The f1 score on the validation set is calculated and compared to the best f1 score so far. If the current f1 score is higher, then the current hyperparameters and corresponding results are updated as the best.
- 4) The function returns the best weights and loss.

On the other hand, we also implement a combination of the grid search method and manual data splitting to find the best model. Grid search is a "brute force" method to test every combination of parameters in every algorithm we use. The split usually provides a good balance between training the model on a sufficiently large dataset and evaluating it on a representative sample of unseen data. Overall, we have the best results with the grid search and manual data splitting method. These will be the results we present in the next subsections.

B. Evaluation Metrics

As evaluation metrics, the F1 score and Accuracy are used to assess the performance of the different methods.

$$\text{F1 Score} = \frac{2TP}{2TP + FP + FN} \tag{1}$$

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + TN + FN} \tag{2}$$

- TP = True Positive, a positive prediction that is correct.
- TN = True Negative, a negative prediction that is correct.
- FP = False Positive, a positive prediction that is incorrect, meaning that the actual class is negative.
- FN = False Negative, a negative prediction that is incorrect, meaning that the actual class is positive.

C. Models Comparison

To demonstrate the importance of data pre-processing and feature selection, we compare the performance of the six different Machine Learning methods we mentioned above, before and after pre-processing. Each method is tuned with different hyperparameters to optimize performance. Table I shows the results comparison before pre-processing. All of the methods evaluated perform poorly. The two MSE methods always predict -1, resulting in high accuracy and a zero F1 score. The other four methods always predict 1, resulting in very low accuracy and a higher F1 score. The overall poor performance highlights the importance of data pre-processing.

Algorithm	Max iter.	γ	λ	Accuracy	F1 score
Least squares	-	-	-	0.086	0.159
MSE GD	50	0.48	-	0.913	0
MSE SG	50	0.48	-	0.913	0
Ridge reg.	-	-	0.094	0.0862	0.159
Logistic reg.	13	0.48	-	0.0862	0.159
Logistic reg. + penalty	13	0.48	0	0.0862	0.159

Table I: Results comparison - Before Pre-Processing

Table II shows the results comparison after pre-processing. All algorithms exhibit improved Accuracy and F1 score after this step. Least squares and ridge regression perform similarly, with ridge regression having a slightly higher accuracy (0.001 higher). Further fine-tuning is needed to determine which method is better. Ridge regression has more potential to perform better because it has a hyperparameter that can be optimized, while least squares does not. Ridge regression with $\gamma = 0.094$ achieved the best Accuracy and F1 scores (Accuracy = 0.867, F1 score = 0.416) after feature selection. This is achieved by tuning the threshold of the prediction function, which assigns a value of 1 or -1 depending on whether the sigmoid output is greater or less than the threshold value. Figure 3 shows the plot of the F1 score as a function of the threshold value. The optimal threshold value that yields the highest F1 score is 0.561.

Algorithm	Max iter.	γ	λ	Accuracy	F1 score
Least squares	-	-	-	0.866	0.416
MSE GD	100	0.48	-	0.086	0.159
MSE SG	100	0.48	-	0.537	0.208
Ridge reg.	-	-	0.094	0.867	0.416
Logistic reg.	13	0.48	-	0.860	0.411
Logistic reg. + penalty	13	0.48	0	0.860	0.411

Table II: Results comparison - After Pre-Processing

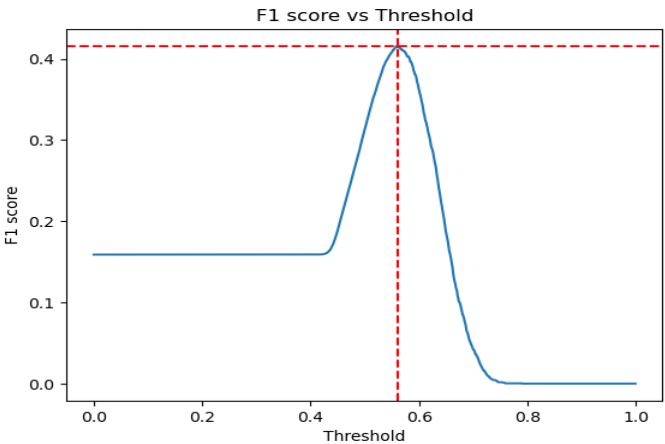
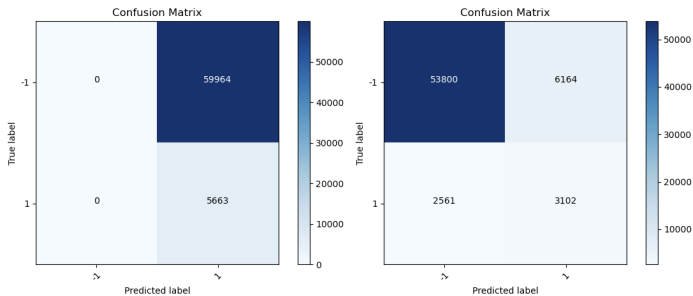


Figure 3: F1 score as a function of the threshold of Predict method

V. BEST MODEL

Using Ridge Regression with $\lambda = 0.094$ and logarithmic transformation, we achieve our best F1 score of 0.417 on the AICrowd leaderboard, which is slightly higher than the score we obtained on the validation set. This demonstrates our ability to effectively tune our model and test its performance accurately on unseen data.



(a) Before Pre-processing (b) After Pre-processing

Figure 4: Confusion Matrices for Ridge Regression

The Confusion Matrices offer a visual explanation of the ridge regression predictions. In the subfigure 4a, before pre-processing, we see that the model chooses to predict all labels to 1 in order to get the best F1-score, since the model can't fit at all. This way, the F1-score is equal to 0.159. The threshold to predict the results is equal to 0, meaning all the predicted labels (that are contained between 0 and 1 since we applied sigmoid) are 1. In the subfigure 4b, after pre-processing, the model fits and therefore the F1-score is much better; 0.41.a

VI. CONCLUSION

In conclusion, we observe the critical role of data pre-processing in improving the performance of Machine Learning models, as evidenced by the significant improvement in the F1 score of all the tested models after pre-processing. However, the F1 score in this project is still not as high as desired due to the extreme imbalance of the data. This dataset provides a valuable introduction to the real-world challenges of Machine Learning, where missing values and unbalanced data are common. Addressing these challenges requires a combination of domain expertise and technical skills to extract the most meaningful insights from the data.

REFERENCES

- [1] CDC BRFSS Survey Data (2015), Centers for Disease Control and Prevention, [available here](#)
- [2] Jason Brownlee, 2020, Discover Feature Engineering, How to Engineer Features and How to Get Good at It, [available here](#)
- [3] Wikipedia, Interquartile range, [available here](#)
- [4] Wikipedia, F1 Score, [available here](#)
- [5] Pedro Domingos, 2012, A Few Useful Things to Know About Machine Learning, [available here](#)